



© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2023

M. Frisbie, *Building Browser Extensions*

https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-8725-5_3

3. Browser Extension Crash Course

Matt Frisbie¹

(1) California, CA, USA

Chapters later in this book will cover various aspects of browser extensions in great detail. This chapter, on the other hand, will guide you through the quickest way to get up and running with browser extensions. The crash course will cover building a trivial browser extension from scratch. It will *not* use any third-party libraries or dependencies, just vanilla HTML, CSS, and JavaScript.

This chapter is geared toward developers that have never built an extension before; however, even if you are already somewhat comfortable with extension development, this chapter may yet prove useful as it touches upon some corners of the domain that you may not have experimented with before.

NoteFor the sake of simplicity, the crash course will only focus on developing a manifest v3 browser extension for Google Chrome. Creating a manifest v2 browser extension, or supporting other browsers, requires steps that are not covered in this crash course.

Creating the Manifest

As described in the previous chapter, the extension manifest primarily defines the following:

- 1.What the extension is allowed to do
- 2.Where its files are located

Begin by creating a new empty directory named `extension-crash-course`. All the files we create in this crash course will be created inside it:

```
extension-crash-course/
```

└─ manifest.json

Inside this directory, let's create the simplest possible `manifest.json` file.

The file structure is shown here:

manifest.json

```
{
  "name": "Extension Crash Course",
  "description": "Browser extension created from scratch",
  "version": "1.0",
  "manifest_version": 3
}
```

These fields define the formal browser extension name string `Extension Crash Course`, and the description string `Browser extension created from scratch`. These strings will be reflected both within the browser itself when the extension is installed, as well as in the Chrome Web Store when the extension is published. Also defined in this file is the semantic version `1.0`, indicating the version of the extension package. It also defines the manifest version string `3`, indicating to the browser how the `manifest.json` file should be parsed.

Minimum Viable Extension

Besides the manifest file, all elements of browser extensions are optional for use. Therefore, this lone `manifest.json` file with its small amount of boilerplate is the minimum viable browser extension. The extension has no features or user interfaces, and it incurs effectively zero browser overhead. It is essentially the browser extension equivalent of a NOOP instruction. Nevertheless, Google Chrome will happily install this single file as a new browser extension just as it would any other.

Installing Your Extension

The fastest way to install and test your extension is to load it into Google Chrome by enabling extension developer mode. This mode can be enabled on your browser's Chrome Extensions page. There are two ways to reach this page:

- **: > More Tools > Extensions (Figure 3-1)**

- Navigate your browser to `chrome://extensions`

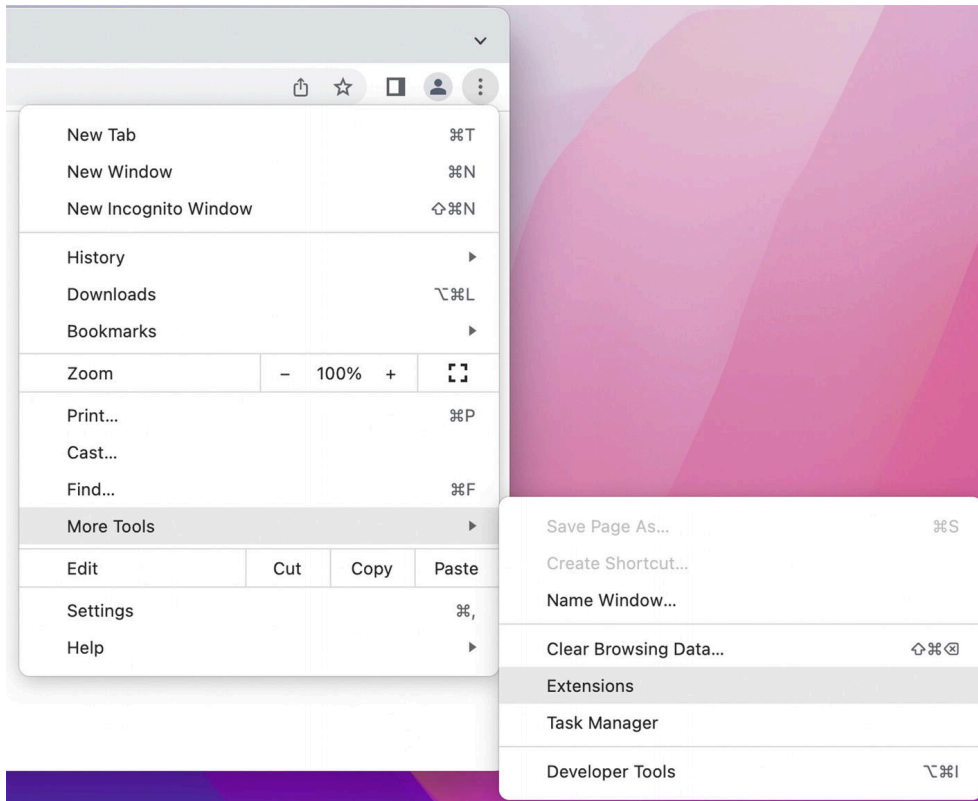


Figure 3-1 Opening the Chrome Extensions page via browser menu

The default behavior of Google Chrome is to disallow loading extensions from the local filesystem. To enable this behavior, on the Chrome Extensions page, you will need to enable the *Developer mode* toggle, shown in Figures [3-2](#) and [3-3](#).

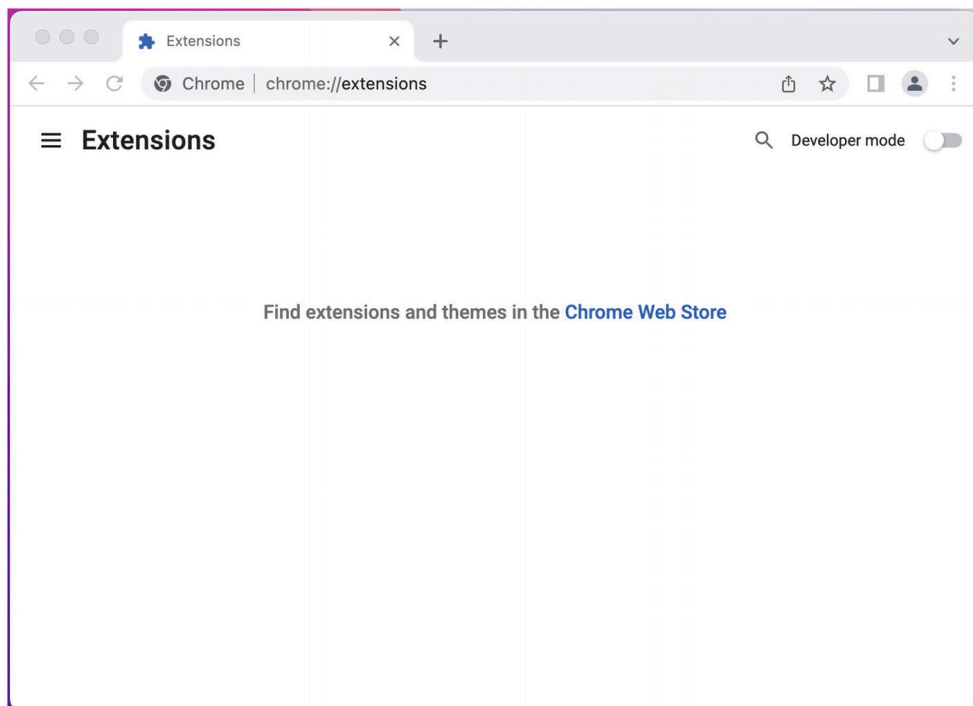


Figure 3-2 The Chrome Extensions page with Developer mode disabled

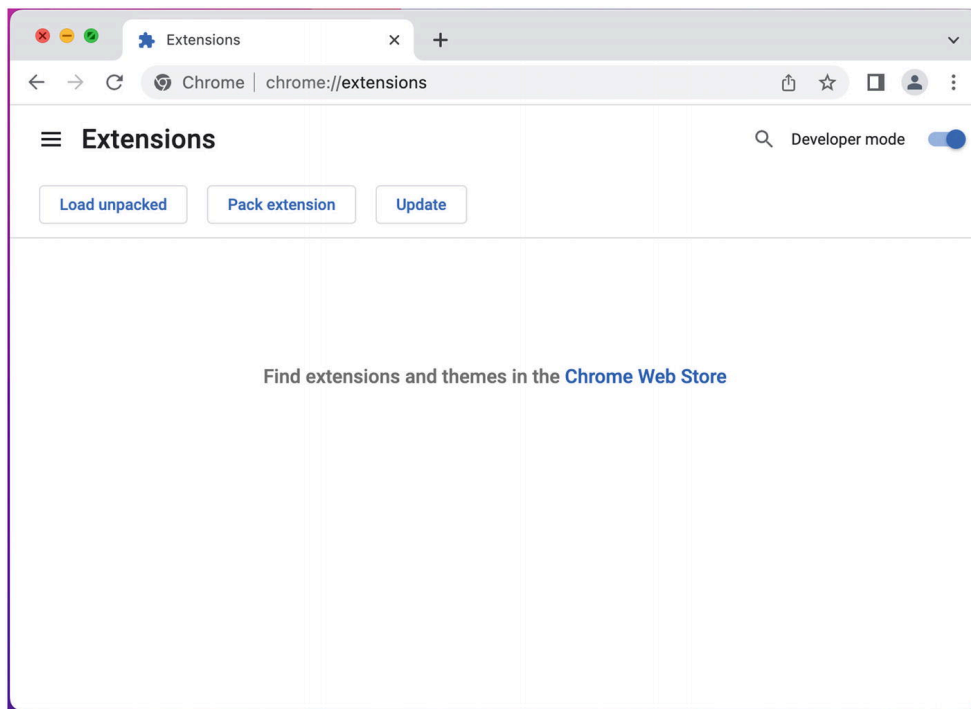


Figure 3-3 The Chrome Extensions page with Developer mode enabled

With Developer mode enabled, you can then load the extension by clicking “Load unpacked” and selecting the `extension-crash-course` directory that contains `manifest.json` (Figure 3-4).

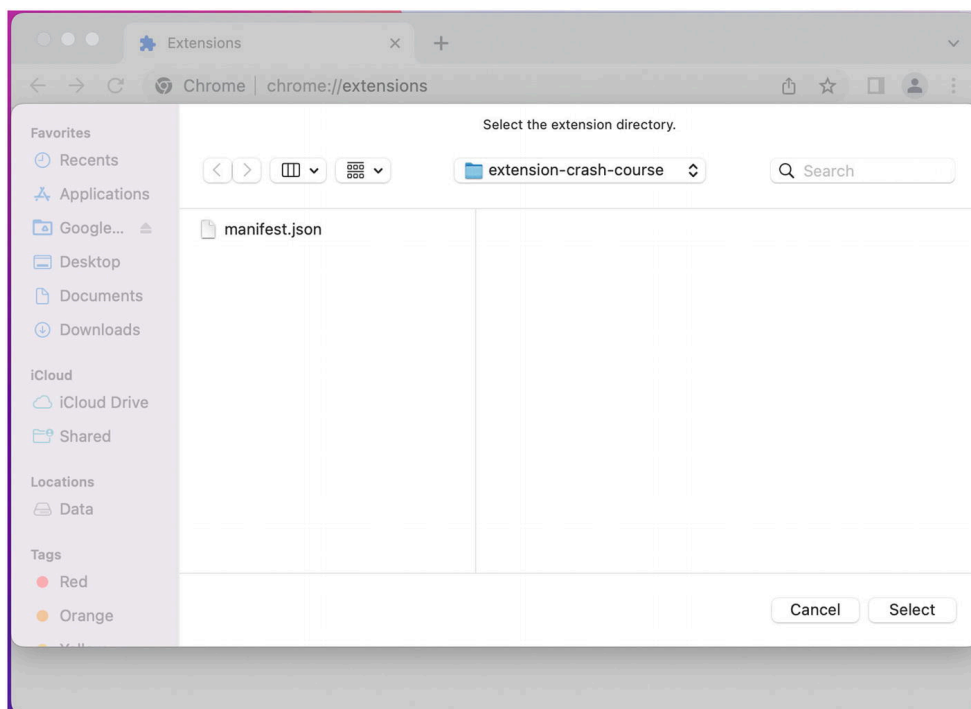


Figure 3-4 Selecting the directory to load the unpacked extension

After selecting the directory, your extension will be installed in the browser and the Chrome Extensions page will reflect this by adding a card for the new extension (Figure 3-5).

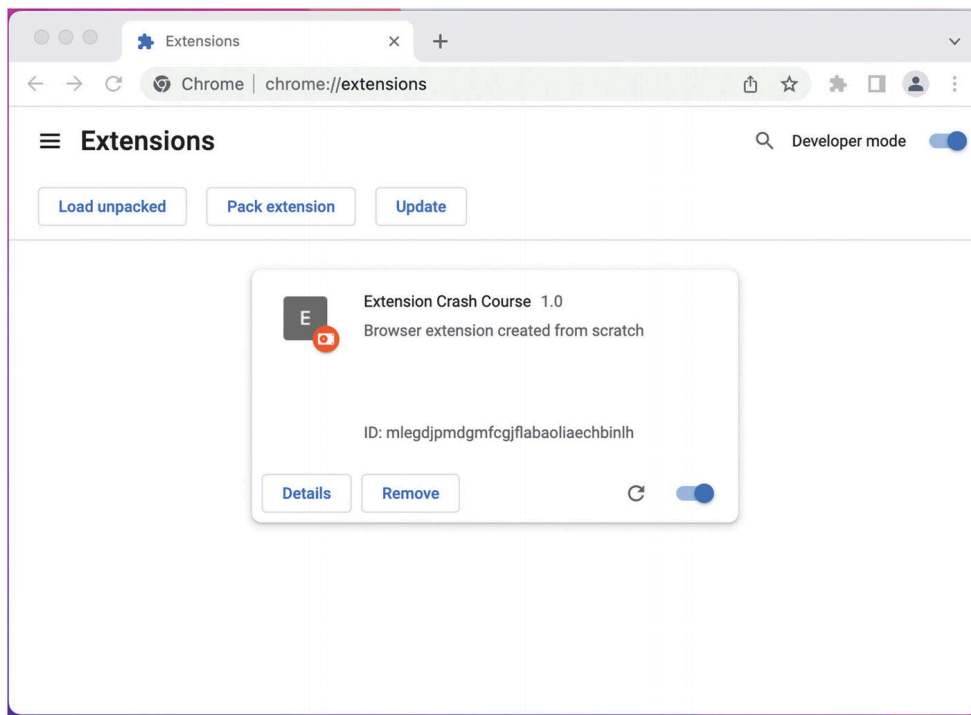


Figure 3-5 Google Chrome with the crash course extension newly installed

Note For ease of access, you should pin the extension toolbar icon so that it always displays. Click on the puzzle piece extension icon, and then click the pin button.

Reloading Your Extension

When making changes to extension code, understanding when those changes should appear can be difficult. Different parts of extensions will be reloaded at different times. Furthermore, multiple tabs and windows mean that you may have multiple versions of an extension running at the same time!

There are several different points at which portions of the extension are reloaded:

- An **extension reload** obtains a new copy of `manifest.json`, updates the background service worker, and closes any stale popups and options pages that are open. This is required when the `manifest.json` changes.
- An **extension page reload** is a page refresh of any page that uses the extension protocol `chrome-extension://`. This is required to reflect changes in HTML, JS, CSS, and images in popup and options pages.

- A **web page reload** is a page refresh of any web page that has content scripts injected. This is required to reflect changes in the injected content scripts.
 - A **devtools reload** is closing and opening the browser's developer tools interface. This is required to reflect changes in devtools pages.
- There are three ways to force an extension reload:

- Uninstall and reinstall the extension
- Click the reload icon ↺ in the corresponding card on the Chrome Extensions page
- Programmatically reload the extension using
`chrome.runtime.reload()` OR `chrome.management.setEnabled()`

Note Programmatically reloading the extension is covered more in the *Extension Development and Deployment* chapter.

When sending messages using the WebExtensions API, you will occasionally see an error similar to the following:

```
Uncaught (in promise) Error: Could not establish connection. Receiving end does not ex
```

Browsers understand that different versions of extensions can be running at the same time, and so they explicitly disallow sending messages between extension parts of different versions. Seeing this error typically indicates that a part of your extension needs to be reloaded.

Note For the purposes of this crash course, reloading the extension *and* the extension page will be sufficient to reflect any changes you have made.

Adding a Background Script

Next, let's add a simple background script and verify that it is running. In the same directory as `manifest.json`, create a `background.js` file:

```
extension-crash-course/  
├─ manifest.json  
└─ background.js
```

The file content is shown below:

background.js

```
console.log('Hello from the background script!');
```

The manifest doesn't know this file exists, so let's update it so it loads this script file as a background service worker:

manifest.json

```
{
  "name": "Extension Crash Course",
  "description": "Browser extension created from scratch",
  "version": "1.0",
  "manifest_version": 3,
  "background": {
    "service_worker": "background.js"
  }
}
```

After reloading the extension, you will notice that the card on the Chrome Extensions page shows a link to the service worker (Figure 3-6):

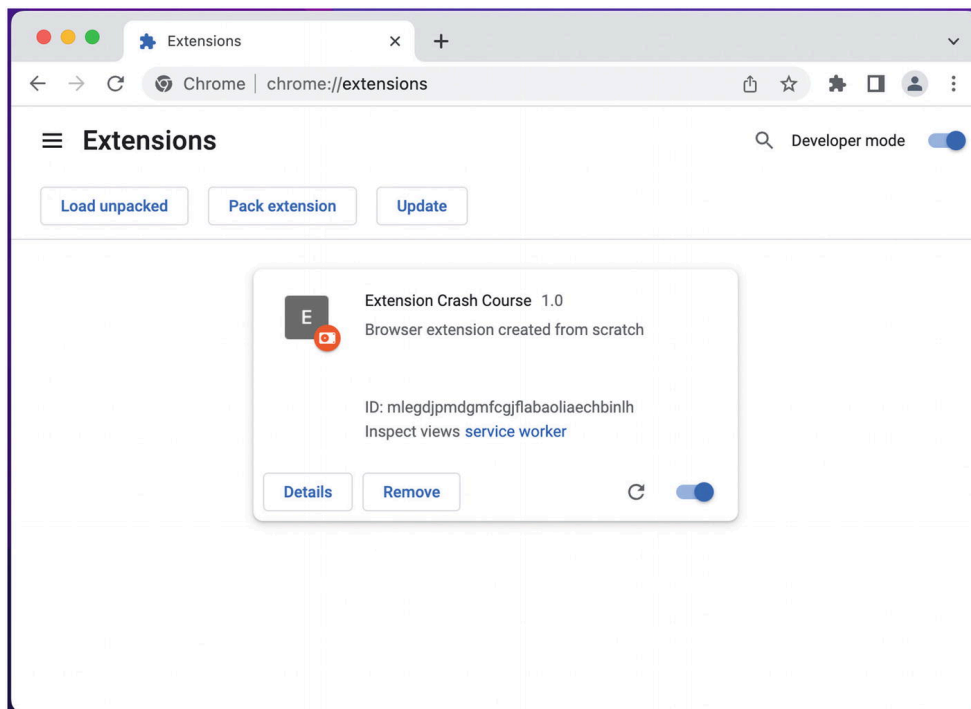


Figure 3-6 Extensions with a link to the service worker

This link will open a developer tools console for the background service worker, where you will see the output of the `console.log` (Figure 3-7).

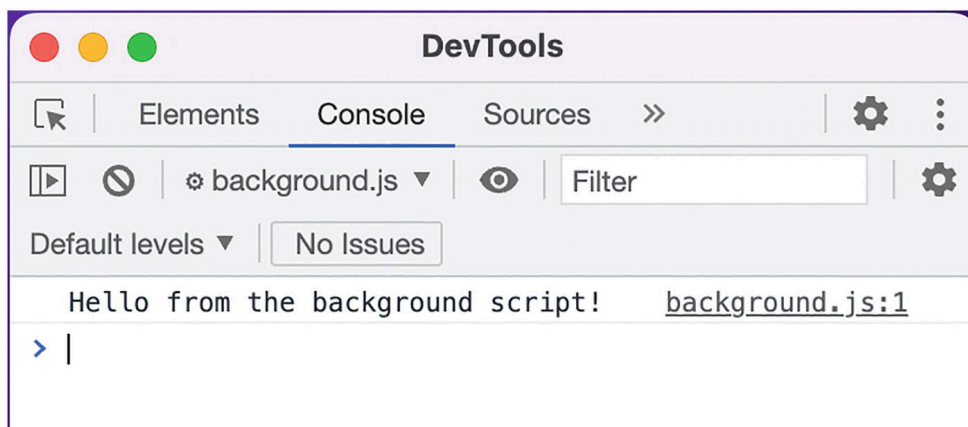


Figure 3-7 Console for the background service worker

Now that the background script is working properly, let's configure it to log event messages it receives via the WebExtensions API:

background.js

```
console.log('Hello from the background script!');
chrome.runtime.onMessage.addListener((msg) => {
  console.log(msg.text);
});
```

Note This listener won't print anything until a message is sent from elsewhere in the browser.

You'll note that the card in the Chrome Extension page may show the service worker as *inactive* after a period of time. In the interest of freeing unused system resources, Google Chrome decided that the service worker was idle and automatically unloaded it. The worker is reloaded when needed again, such as when an extension message is sent.

Note Unloading and reloading service workers is an important concept. It will be covered further in the *Background Scripts* chapter.

Adding a Popup Page

Next, let's add the first user interface to the extension. Create a new `popup` directory in the same directory as `manifest.json`. Inside the new directory, create three new files: `popup.html`, `popup.css`, and `popup.js`:

```
extension-crash-course/
├─ manifest.json
├─ background.js
```



```
└─ popup/  
  └─ popup.html  
  └─ popup.css  
  └─ popup.js
```

The file contents are shown below:

popup/popup.html

```
<!DOCTYPE html>  
<html>  
  <head>  
    <link href="popup.css" rel="stylesheet" />  
  </head>  
  <body>  
    <h1>This is the popup page!</h1>  
    <script src="popup.js"></script>  
  </body>  
</html>
```

popup/popup.css

```
body {  
  width: 400px;  
  margin: 2rem;  
}
```

popup/popup.js

```
console.log('Hello from the popup!');
```

The manifest needs to be told when to render this HTML, so let's update it so it opens the popup page when the toolbar icon is clicked:

manifest.json

```
{  
  "name": "Extension Crash Course",  
  "description": "Browser extension created from scratch",  
  "version": "1.0",  
  "manifest_version": 3,  
  "background": {  
    "service_worker": "background.js"
```

```
},  
"action": {  
  "default_popup": "popup/popup.html"  
}  
}
```

After reloading the extension, click the toolbar extension icon to open the popup page (Figure 3-8).

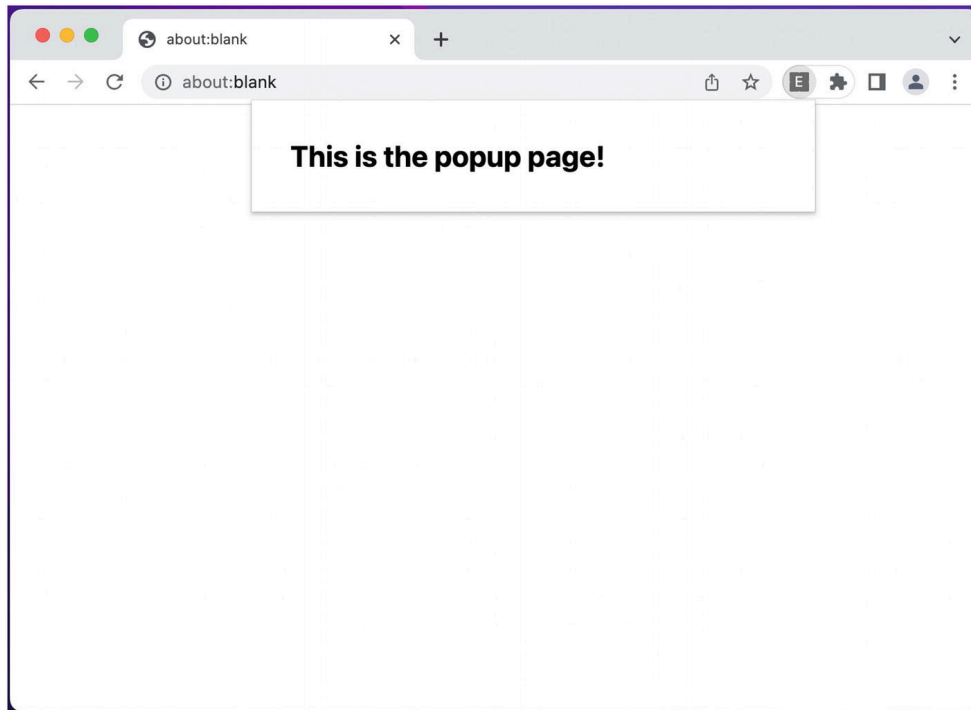


Figure 3-8 Opened popup page

Note Because you have not defined an extension icon, Google Chrome will automatically generate one from the name of the extension. As shown here, the default is the first letter of the extension name, E, on a gray background.

Next, let's send a message from the popup to the background script. Update the following files:

popup/popup.html

```
<!DOCTYPE html>  
<html>  
  <head>  
    <link href="popup.css" rel="stylesheet" />  
  </head>  
  <body>  
    <h1>This is the popup page!</h1>
```

```
<button id="btn">Send popup message</button>
<script src="popup.js"></script>
</body>
</html>
```

popup/popup.js

```
console.log('Hello from the popup!');
document.querySelector("#btn").addEventListener('click', () => {
  chrome.runtime.sendMessage({ text: "Popup" });
});
chrome.runtime.onMessage.addListener((msg) => {
  document.body.innerHTML += `<div>${msg.text}</div>`;
});
```

This new code adds a button to the popup that sends a message to the rest of the extension. It also adds a listener for incoming messages. Reload the extension, click the new button in the popup, and inspect the background script to see the message logged (Figure 3-9).

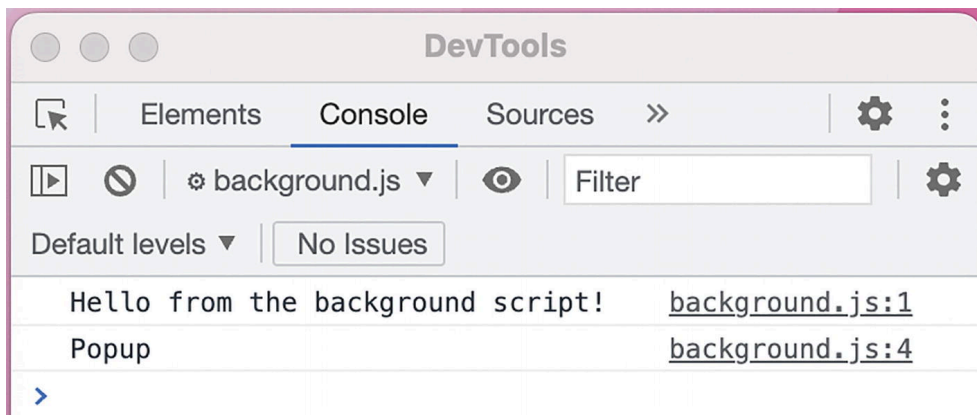


Figure 3-9 Background script logs the message from the popup page

Note that, even though we set a listener for messages in the popup page, it did not handle an incoming message. Although the extension message infrastructure works as a broadcast, the source that sends the message will not also receive it.

Adding an Options Page

Next, let's add an options page with a similar structure to the popup page:

```
extension-crash-course/
├─ manifest.json
```

```
├─ background.js
├─ popup/
│   ├─ popup.html
│   ├─ popup.css
│   └─ popup.js
└─ options/
    ├─ options.html
    ├─ options.css
    └─ options.js
```

The file contents are shown below:

options/options.html

```
<!DOCTYPE html>
<html>
  <head>
    <link href="options.css" rel="stylesheet" />
  </head>
  <body>
    <h1>This is the options page!</h1>
    <button id="btn">Send options message</button>
    <script src="options.js"></script>
  </body>
</html>
```

options/options.css

```
body {
  margin: 2rem;
}
```

options/options.js

```
console.log('Hello from options!');
document.querySelector("#btn").addEventListener('click', () => {
  chrome.runtime.sendMessage({ text: "Options" });
});
chrome.runtime.onMessage.addListener((msg) => {
  document.body.innerHTML += `<div>${msg.text}</div>`;
});
```

The manifest needs to be configured to use this HTML as the options page:

manifest.json

```
{
  "name": "Extension Crash Course",
  "description": "Browser extension created from scratch",
  "version": "1.0",
  "manifest_version": 3,
  "background": {
    "service_worker": "background.js"
  },
  "action": {
    "default_popup": "popup/popup.html"
  },
  "options_page": "options/options.html"
}
```

Reload the extension, and open the extension options page by right clicking on the toolbar icon and selecting “Options” (Figure [3-10](#)).

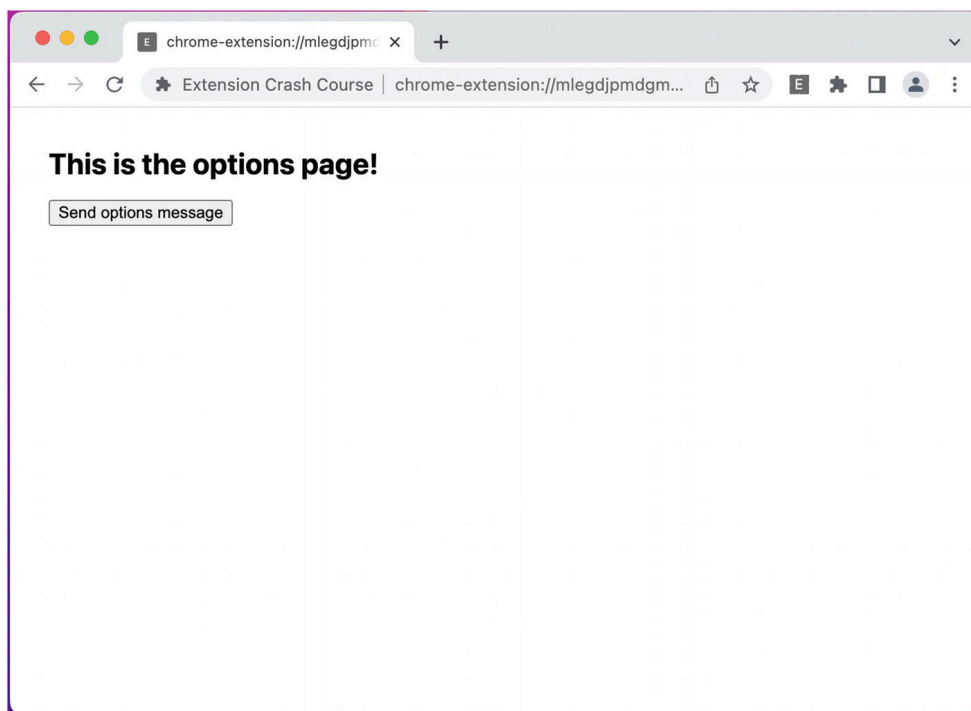


Figure 3-10 Extension options page

To test the messaging system between extension components, open the popup page and click its button. The message handler on the options page will add the message into the page (Figure [3-11](#)).

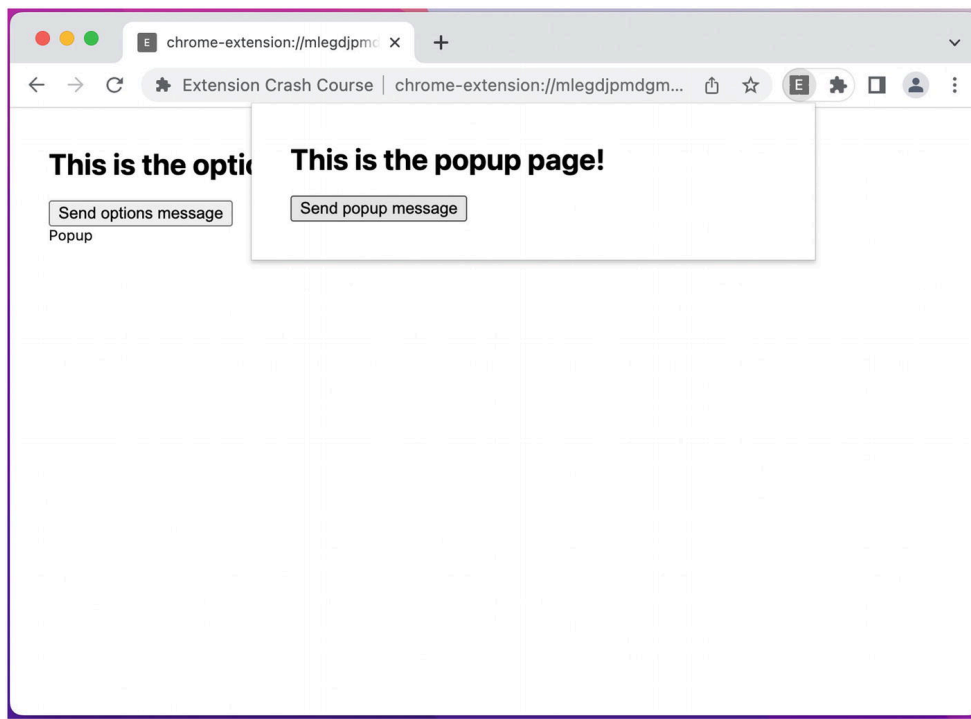


Figure 3-11 Extension options page with popup message

Adding a Content Script

Next, let's create some content scripts that add similar behavior to the options and popup pages:

```
extension-crash-course/  
├─ manifest.json  
├─ background.js  
├─ popup/  
│   ├─ popup.html  
│   ├─ popup.css  
│   └─ popup.js  
├─ options/  
│   ├─ options.html  
│   ├─ options.css  
│   └─ options.js  
└─ content-scripts/  
    ├─ content-script.css  
    └─ content-script.js
```

The content scripts will add a styled custom container into the page:

content-scripts/content-script.css

```
#container {
  position: absolute;
  background-color: gray;
  color: white;
  padding: 2rem;
  top: 0;
  left: 0;
}
```

content-scripts/content-script.js

```
console.log('Hello from content script!');
document.body.innerHTML += `
<div id="container">
  <h1>This is the content script!</h1>
  <button id="btn">Send content script message</button>
</div>
`;
document.querySelector("#btn").addEventListener('click', () => {
  chrome.runtime.sendMessage({ text: "Content script" });
});
chrome.runtime.onMessage.addListener((msg) => {
  document.querySelector('#container').innerHTML += `<div>${msg.text}</div>`;
});
```

Configure the manifest to inject these content scripts into all valid web pages:

manifest.json

```
{
  "name": "Extension Crash Course",
  "description": "Browser extension created from scratch",
  "version": "1.0",
  "manifest_version": 3,
  "background": {
    "service_worker": "background.js"
  },
  "action": {
    "default_popup": "popup/popup.html"
  },
  "options_page": "options/options.html",
  "content_scripts": [
    {
```

```
"matches": ["<all_urls>"],  
"css": ["content-scripts/content-script.css"],  
"js": ["content-scripts/content-script.js"]  
}  
]  
}
```

Open a real web page such as <https://blank.org> to see the content script render (Figure 3-12):

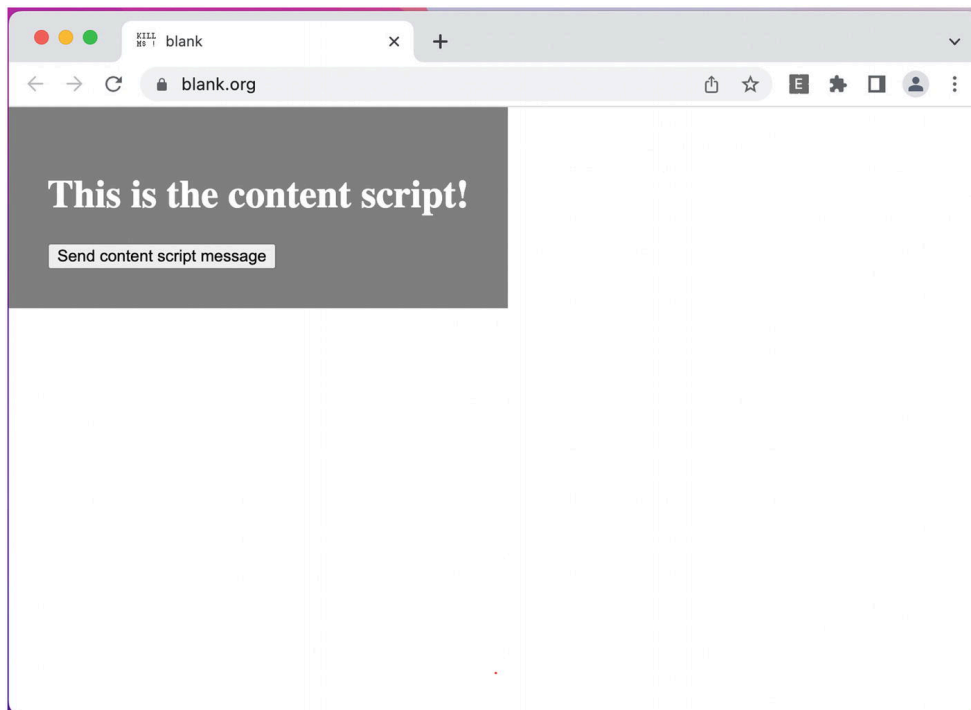


Figure 3-12 Content script running on web page

In this state, you will be able to click the content script button and have the messages show up in the extension views, such as on the options page.

Working with Multiple Tabs

After experimenting with the extension in this state, you will discover that the messages can be successfully sent from the content scripts to the other parts of the extension, but messages are *not* received in the content script.

This is because content scripts are inherently bound to the browser's tab model. There may be many tabs open all running the same content script, so there must be consideration as to which content script instances

should receive the message. To send a message to a content script, the WebExtensions API allows you to individually target tabs. Update the popup script to the following:

popup/popup.js

```
console.log('Hello from the popup!');
document.querySelector("#btn").addEventListener('click', () => {
  chrome.runtime.sendMessage({ text: "Popup" });
  chrome.tabs.query({
    active: true,
    currentWindow: true
  }, (tabs) => {
    chrome.tabs.sendMessage(
      tabs[0]?.id,
      { text: "Popup" }
    );
  });
});
chrome.runtime.onMessage.addListener((msg) => {
  document.body.innerHTML += `<div>${msg.text}</div>`;
});
```

After reloading the extension, you should now see that messages sent from the popup page's button will show up in the content script view, but *only* on the active tab.

Tip Before continuing, experiment with the extension in its current state. Open multiple tabs, multiple browser windows, multiple popups in different windows, and send messages from different locations to see where they do or do not show up. The results may surprise you. This will be an instructive lesson on how the broadcast messaging model works for extensions.

Adding a Devtools Panel

Next, let's add a panel to the Chrome developer tools. Create the following files:

```
extension-crash-course/
├─ manifest.json
├─ background.js
├─ popup/
```

```
|   ├── popup.html
|   ├── popup.css
|   └── popup.js
├── options/
|   ├── options.html
|   ├── options.css
|   └── options.js
├── content-scripts/
|   ├── content-script.css
|   └── content-script.js
└── devtools/
    ├── devtools.html
    ├── devtools.js
    ├── devtools_panel.html
    └── devtools_panel.js
```

Adding devtools interfaces is slightly different from other extension interfaces. The extension is provided a top-level headless page that uses the WebExtensions API in a script to insert custom developer tools interfaces. The files should appear as follows:

devtools/devtools.html

```
<!DOCTYPE html>
<html>
  <body>
    <script src="devtools.js"></script>
  </body>
</html>
```

devtools/devtools.js

```
chrome.devtools.panels.create(
  "Devtools Panel",
  "",
  "/devtools/devtools_panel.html"
);
```

Next, configure the devtools panel to log the URL of all outgoing network activity from the page using the Devtools API:

devtools/devtools_panel.html

```
<!DOCTYPE html>
<html>
  <body>
    <h1>This is the devtools panel!</h1>
    <script src="devtools_panel.js"></script>
  </body>
</html>
```

devtools/devtools_panel.js

```
console.log("Hello from the devtools panel!");
chrome.devtools.network.onRequestFinished.addListener(
  (request) => {
    document.body.innerHTML +=
      `<div>${request.request.url}</div>`;
  }
);
```

Finally, update the manifest to use the devtools headless page:

manifest.json

```
{
  "name": "Extension Crash Course",
  "description": "Browser extension created from scratch",
  "version": "1.0",
  "manifest_version": 3,
  "background": {
    "service_worker": "background.js"
  },
  "action": {
    "default_popup": "popup/popup.html"
  },
  "options_page": "options/options.html",
  "devtools_page": "devtools/devtools.html",
  "content_scripts": [
    {
      "matches": ["<all_urls>"],
      "css": ["content-scripts/content-script.css"],
      "js": ["content-scripts/content-script.js"]
    }
  ]
}
```

You'll need to reload the extension as well as close and reopen any open developer tools windows to see the changes. Visiting any web page will log its traffic (Figure [3-13](#)).

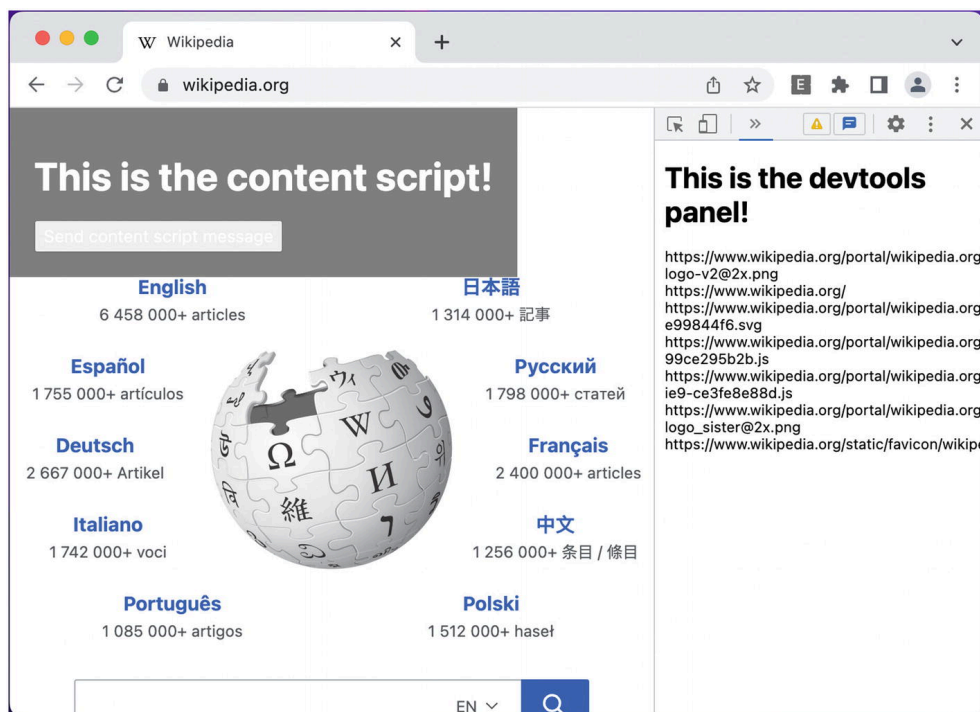


Figure 3-13 Devtools panel logging network traffic

Summary

In this chapter, you were guided through the process of creating a simple Chrome extension. The purpose of this crash course was to provide step-by-step instructions on how to create an extension that touched all the different user interface elements that browser extensions can use. After completing the crash course, you should have a good understanding of how various user interfaces are produced from their source code, as well as how a manifest should be laid out to achieve these ends.

You should also have a very basic understanding of how the WebExtensions API can be used. This crash course only touched upon a handful of basic methods, but it gives you an initial concept for all the various ways that browser extensions can manipulate the browser to achieve their ends.

The next chapter will explore the architecture of browser extensions, including how files are organized and how the various elements behave inside the browser.

